Bazar.com: Design and Evaluation

Distributed and Operating Systems - Lab 1

1. Introduction

Bazar.com is a small bookstore developed to study service communication, concurrent requests, and persistent state in a distributed system. It supports four predefined books, divided equally between distributed systems and undergraduate school. Users search by topic, inspect a book by ID, and purchase one copy per request. The two-tier architecture places a Frontend gateway ahead of Catalog and Order services. A command-line client runs on the host. This structure makes ownership and failure boundaries visible while keeping the application small enough to inspect and test.

2. System Architecture

Three independent Flask applications exchange JSON over HTTP REST. Frontend publishes port 5000, Catalog port 5001, and Order port 5002. Frontend forwards search and information requests to Catalog and purchase requests to Order. Order consults Catalog and requests inventory changes. Only Catalog opens the books database; only Order opens the purchase-attempt database.

```
Host CLI --HTTP--> Frontend :5000       [entry tier]
                     |             \
                search/info       purchase
                     v               v
                 Catalog :5001 <-- Order :5002  [backend tier]
                     |               |
               catalog.sqlite3   order.sqlite3
```

`compose.yaml` builds exactly three Linux containers from `python:3.12-slim`. Each runs Gunicorn with one worker and four threads. Compose DNS supplies the `catalog` and `order` hostnames. Separate named volumes, `catalog_data` and `order_data`, mount at `/data`; Frontend has no database volume. Environment variables configure service URLs and database paths without embedding machine addresses in request handlers.

3. Implementation and Request Flow

Catalog implements `GET /search/<topic>` and `GET /info/<id>`. Search trims surrounding whitespace and compares topics without ASCII case sensitivity. Unknown topics return an empty array; missing IDs return 404. Initialization inserts missing seed IDs with quantity 10 and price 10.00, preserving existing records. `PATCH /update/<id>` accepts a replacement price, an integer `quantity_delta`, or both. Validation rejects unsupported fields, booleans, invalid numeric types, negative prices, and excess decimal precision. Rejected combined updates preserve both original fields.

For `POST /purchase/<id>`, Order first validates Catalog's information response. It commits a UUID-based `pending` attempt before requesting a one-copy decrement. The initial lookup is not a reservation: the decrement determines availability. Recognized Catalog rejections produce a `failed` attempt; successful decrements are followed by `completed`. The completed title and price snapshot come from the decrement response. Lost or malformed responses produce `unknown` when that status can be saved; interruption can leave `pending`.

Frontend validates successful upstream responses and sanitizes error messages while preserving meaningful statuses and valid attempt IDs. Typical outcomes are 201 for completion, 400 for invalid input, 404 for a missing book, 409 for insufficient stock, and 502/503 for upstream failures. The CLI prints readable search results and book details, and successful purchase output includes `bought book <book_name>`. Failures exit nonzero and purchase warnings discourage automatic replay. Connect/read timeouts are explicit: Order uses `(2, 10)`, Frontend `(2, 45)`, and the CLI `(2, 60)` seconds.

4. Concurrency and Data Persistence

Catalog's update handler starts `BEGIN IMMEDIATE` and executes a guarded SQL update within one transaction. A decrement succeeds only when the current stock is sufficient; overflow guards also protect additions. SQLite serializes writers, so concurrent requests cannot independently consume the same last copy. Price and quantity changes commit together or roll back together. Each request uses its own connection with a five-second SQLite busy timeout.

Catalog stores nonnegative integer quantities and integer-cent prices. Order stores attempt IDs, book details, price snapshots, timestamps, and outcome states. Startup preserves existing data, and named volumes retain both databases across container restarts. However, the pending insert, Catalog decrement, and completion write are separate transactions. The full purchase is not a distributed transaction: a decrement can commit even when its response or the subsequent Order write fails.

5. Testing and Evaluation

The recorded Step 4 run passed 14 of 14 automated tests in 2.512 seconds, with no failures or skips. Tests cover queries, update validation, combined-update rejection, purchases, persistence initialization, CLI behavior, and malformed or ambiguous upstream responses. Host tests use temporary SQLite files and focused mocks; separate Docker checks exercised real network communication.

All three images built, exactly three services ran, and every `/health` endpoint returned HTTP 200. With five copies available, twelve concurrent purchases through the Docker Frontend produced five completed purchases and seven out-of-stock responses. Final stock was zero, and successful response IDs matched the five persisted completed orders. Restarting all services preserved book 3 at price 17.25 and quantity 13, together with its completed attempt.

Stopping Catalog produced structured 503 errors for search, information, and purchase, followed by successful recovery. Stopping Order left search and information available; purchase failed conservatively and stock remained unchanged. Recovery restored purchases. Real CLI output was captured, and a scan of 282 service-log lines found zero internal-error matches. Detailed evidence is in verification.md and sample-output.txt.

6. Design Decisions and Trade-offs

Flask keeps routing and validation explicit without the structure of a larger framework. SQLite avoids a separate database server and suits four books, but serializes writes. Three services demonstrate independent ownership and failure handling at the cost of network calls and coordination compared with a monolith. Synchronous REST is straightforward to trace, although slow dependencies occupy request threads. Integer-cent storage preserves ordinary monetary calculations inside the databases; floating-point JSON serialization can lose precision for very large prices. These choices favor understandable lab behavior over high throughput and horizontal scalability.

7. Known Limitations and Improvements

No cross-service transaction, automatic reconciliation, or purchase idempotency key is implemented. Pending and unknown attempts require administrative inspection; the system neither retries decrements nor blindly restores stock. SQLite write contention can return temporary failures. Health checks test local processes, not database or dependency readiness. Authentication, richer monitoring, scalable storage, and a carefully designed recovery mechanism are future improvements, not current features.

8. How to Run

Install Docker with Compose and Linux-container support, plus host Python and the client dependency. From the repository root, run:

```
docker compose up --build -d
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r client/requirements.txt
.\.venv\Scripts\python.exe client/client.py search "distributed systems"
.\.venv\Scripts\python.exe client/client.py info 1
.\.venv\Scripts\python.exe client/client.py purchase 1
docker compose logs --tail 100
docker compose down
```

Wait for all health endpoints before requests. Shutdown retains database volumes. The README provides full configuration and testing instructions.